

Materi Pembelajaran

Portal Manajemen Proyek TI

Materi pemahaman untuk orang awam: konsep dasar website, cara kerja kode, fitur aplikasi, dan cara menggunakan sistem.

Tujuan materi: setelah membaca dokumen ini, pembaca diharapkan memahami apa itu frontend, backend, database, API, login, role akses, serta bagaimana fitur portal ini dibuat dan digunakan.

1. Gambaran Besar Website

Website ini adalah portal manajemen proyek untuk membantu tim konsultan TI memantau proyek, tugas, milestone, tim, risiko, dokumen, komentar, dan waktu kerja.

Secara sederhana, website ini bekerja seperti sistem pencatatan dan pemantauan. Pengguna login, lalu melihat dashboard. Data yang tampil di dashboard diambil dari database melalui backend API.

Masalah yang Dibantu oleh Website

- Mengumpulkan data proyek dalam satu tempat.
- Memantau status proyek dan tugas agar progres terlihat jelas.
- Membantu Project Manager melihat risiko keterlambatan.
- Menyediakan catatan komunikasi dan tautan dokumen proyek.
- Membatasi akses berdasarkan peran pengguna.

Ringkasan Teknologi

Bagian	Teknologi	Fungsi
Frontend	React, Vite, CSS, Chart.js	Menampilkan halaman login, dashboard, form, tabel, filter, dan grafik.
Backend	Node.js, Express.js	Menyediakan API untuk login dan pengelolaan data.
Database	MySQL	Menyimpan user, proyek, tugas, milestone, tim, risiko, file, komentar, dan log waktu.
Keamanan	JWT, bcrypt, Helmet, CORS, Zod	Login aman, password hash, validasi input, dan pembatasan akses.
Deployment lokal	Docker Compose, Nginx	Menjalankan frontend, backend, dan database dengan satu perintah.

2. Konsep Dasar untuk Orang Awam

2.1. Apa Itu Website?

Website adalah aplikasi yang dibuka lewat browser seperti Chrome, Edge, atau Firefox. Browser menampilkan tampilan, tombol, form, tabel, dan grafik yang bisa digunakan pengguna.

2.2. Apa Itu Frontend?

Frontend adalah bagian yang dilihat pengguna. Di project ini, frontend berada di folder `frontend/`. Contohnya halaman login, dashboard, daftar proyek, tombol tambah, tombol edit, dan grafik.

2.3. Apa Itu Backend?

Backend adalah bagian server yang bekerja di belakang layar. Backend menerima permintaan dari frontend, mengecek login, mengambil data dari database, menyimpan perubahan, dan mengirim jawaban kembali ke frontend.

2.4. Apa Itu Database?

Database adalah tempat penyimpanan data. Jika website diibaratkan kantor, database adalah lemari arsipnya. Semua data penting seperti akun, proyek, tugas, risiko, dan komentar disimpan di MySQL.

2.5. Apa Itu API?

API adalah jalur komunikasi antara frontend dan backend. Frontend tidak mengambil data langsung dari database. Frontend meminta data ke backend lewat alamat API seperti `/api/projects` atau `/api/tasks`.

2.6. Apa Itu CRUD?

Istilah	Arti	Contoh di Website
Create	Membuat data baru	Tambah proyek atau tambah tugas.
Read	Membaca data	Melihat daftar proyek di dashboard.
Update	Mengubah data	Edit status tugas menjadi selesai.
Delete	Menghapus data	Hapus data risiko yang tidak dipakai.

3. Struktur Folder Project

Project ini dipisahkan agar mudah dipahami dan dirawat.

Folder/File	Isi	Dipakai Untuk
frontend/	Kode React, CSS, konfigurasi Vite, dan Dockerfile frontend.	Membuat tampilan website.
backend/	Kode Express, route API, middleware, validator, koneksi database.	Mengatur proses login, API, dan logika server.
backend/src/routes/	File route seperti <code>projects.js</code> , <code>tasks.js</code> , <code>risks.js</code> .	Menentukan endpoint API untuk setiap fitur.
backend/src/middleware/	Middleware autentikasi dan error handling.	Mengecek token dan menangani error.
backend/src/validators/	Skema validasi Zod.	Memastikan data input sesuai aturan.
database.sql	Schema dan data demo MySQL.	Membuat tabel dan isi awal database.
docker-compose.yml	Konfigurasi MySQL, backend, dan frontend.	Menjalankan semua service sekaligus.
docs/	Dokumentasi project.	Referensi deployment, API, proposal, dan keamanan.

4. Arsitektur Cara Kerja Aplikasi

Alur kerja aplikasi bisa dibaca seperti ini:

```
Browser pengguna
-> Frontend React
-> API Backend Express
-> Database MySQL
-> Backend mengirim data
-> Frontend menampilkan data
```

Contoh Saat Pengguna Melihat Daftar Tugas

1. Pengguna membuka dashboard.
2. Frontend menjalankan fungsi untuk mengambil data.
3. Frontend memanggil API `GET /api/tasks`.
4. Backend mengecek token login.
5. Backend mengambil data dari tabel `tasks` dan tabel terkait.
6. Backend mengirim data dalam format JSON.

7. Frontend menampilkan data dalam tabel, kartu, dan grafik.

5. Penjelasan Frontend

5.1. Entry Point

File `frontend/src/main.jsx` adalah pintu masuk aplikasi React. File ini memasang `BrowserRouter` untuk navigasi dan `AuthProvider` untuk status login.

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <BrowserRouter>  
    <AuthProvider>  
      <App />  
    </AuthProvider>  
  </BrowserRouter>  
);
```

5.2. Routing Aplikasi

File `frontend/src/App.jsx` mengatur halaman apa yang muncul berdasarkan URL. Halaman `/login` bisa dibuka tanpa login, sedangkan dashboard dilindungi oleh `ProtectedRoute`.

Route	Fungsi
<code>/login</code>	Menampilkan halaman login.
<code>/</code>	Menampilkan dashboard setelah berhasil login.
<code>/projects</code> , <code>/tasks</code> , <code>/milestones</code> , <code>/teams</code>	Mengarahkan pengguna ke bagian tertentu di dashboard.

5.3. AuthContext

`AuthContext.jsx` menyimpan informasi siapa yang sedang login. Token dan data user disimpan di `localStorage` agar pengguna tidak langsung logout saat halaman direfresh.

Fungsi utamanya adalah `login()` dan `logout()`.

5.4. API Client

`frontend/src/api/api.js` adalah pembungkus `fetch`. Tujuannya agar semua request ke backend memakai cara yang sama. Jika ada token login, file ini otomatis menambahkan header `Authorization: Bearer token`.

```
const token = localStorage.getItem('project_portal_token');  
headers.Authorization = `Bearer ${token}`;
```

5.5. Dashboard

`Dashboard.jsx` adalah file frontend terbesar karena memuat banyak fitur: ringkasan proyek, tugas, milestone, tim, grafik, risiko, file, komentar, form tambah/edit, filter, dan export CSV.

Dashboard mengambil data secara paralel dari banyak endpoint, seperti `/projects`, `/tasks`, `/milestones`, `/teams`, `/risks`, `/time-logs`, dan lainnya.

6. Penjelasan Backend

6.1. Express App

File `backend/src/app.js` mengatur server utama. Di sana dipasang middleware keamanan seperti `helmet`, `cors`, body parser, logging request, route API, dan error handler.

6.2. Koneksi Database

File `backend/src/db.js` membuat connection pool MySQL. Pool dipakai agar server bisa menjalankan banyak query dengan lebih efisien.

Environment Variable	Fungsi
<code>DB_HOST</code>	Alamat server database.
<code>DB_PORT</code>	Port MySQL, biasanya 3306.
<code>DB_NAME</code>	Nama database, yaitu <code>project_portal</code> .
<code>DB_USER</code>	User database.
<code>DB_PASSWORD</code>	Password database.

6.3. Route API

Setiap fitur mempunyai file route sendiri. Contohnya fitur proyek ada di `projects.js`, tugas di `tasks.js`, risiko di `risks.js`, dan komentar di `taskComments.js`.

File Route	Endpoint Utama	Fitur
<code>auth.js</code>	<code>/api/auth/login</code>	Login dan pembuatan token.
<code>users.js</code>	<code>/api/users</code>	Data pengguna.
<code>projects.js</code>	<code>/api/projects</code>	CRUD proyek.
<code>tasks.js</code>	<code>/api/tasks</code>	CRUD tugas dan update progres.
<code>milestones.js</code>	<code>/api/milestones</code>	Milestone proyek.
<code>teams.js</code>	<code>/api/teams</code>	Tim dan anggota tim.
<code>projectLinks.js</code>	<code>/api/project-links</code>	Tautan repository, BRD, staging, dan dokumen lain.
<code>risks.js</code>	<code>/api/risks</code>	Risk register.
<code>timeLogs.js</code>	<code>/api/time-logs</code>	Pencatatan jam kerja.
<code>projectFiles.js</code>	<code>/api/project-files</code>	Repository file proyek.
<code>taskComments.js</code>	<code>/api/task-comments</code>	Komentar tugas.

6.4. Validasi Data

File `backend/src/validators/schemas.js` memakai Zod untuk memastikan data yang masuk benar. Misalnya password minimal 8 karakter, email harus valid, URL harus valid, dan progress tugas harus 0 sampai 100.

7. Login, Token, dan Hak Akses

7.1. Alur Login

1. Pengguna mengisi username dan password di halaman login.
2. Frontend mengirim data ke `POST /api/auth/login`.
3. Backend mencari username di tabel `users`.
4. Password yang diketik dibandingkan dengan password hash memakai `bcrypt.compare`.
5. Jika benar, backend membuat JWT.
6. Frontend menyimpan token di `localStorage`.
7. Setiap request berikutnya membawa token tersebut.

7.2. Apa Itu JWT?

JWT adalah token login. Token ini seperti kartu akses digital. Backend bisa membaca token untuk mengetahui ID user, username, dan role pengguna.

7.3. Role Based Access Control

Hak akses diatur berdasarkan role. Di kode, role disimpan sebagai `pm`, `dev`, dan `client`. Dalam tampilan materi ini, `client` dapat dipahami sebagai pengguna proyek.

Role di Kode	Makna	Akses Umum
<code>pm</code>	Project Manager	Mengelola proyek, tugas, milestone, tim, risiko, link, file, dan data utama.
<code>dev</code>	Developer	Melihat tugas terkait, memperbarui status/progres tugas, mengisi log waktu, dan memberi komentar.
<code>client</code>	Pengguna proyek	Melihat informasi proyek/milestone terkait dan memberi komentar sesuai akses.

8. Database dan Tabel Utama

Database bernama `project_portal`. Tabel dibuat dari file `database.sql`.

Tabel	Isi Data	Hubungan Penting
<code>users</code>	Akun pengguna, email, password hash, role.	Dipakai oleh proyek, tugas, komentar, log waktu, dan risiko.
<code>projects</code>	Nama proyek, deskripsi, tanggal, status, PM, pengguna proyek.	Menjadi induk dari tugas, milestone, risiko, link, dan file.
<code>tasks</code>	Tugas, status, progress, penanggung jawab, due date.	Terhubung ke proyek dan user.
<code>milestones</code>	Titik pencapaian penting proyek.	Terhubung ke proyek.
<code>teams</code> dan <code>team_members</code>	Tim dan anggota tim.	<code>team_members</code> menghubungkan tim dengan user.
<code>time_logs</code>	Catatan jam kerja.	Terhubung ke user dan tugas.
<code>risks</code>	Risiko proyek, peluang, dampak, mitigasi, status.	Terhubung ke proyek dan pemilik risiko.
<code>project_links</code>	Tautan BRD, API docs, repository, staging, dan lainnya.	Terhubung ke proyek.
<code>project_files</code>	Metadata file atau link dokumen proyek.	Terhubung ke proyek dan user pengunggah.
<code>task_comments</code>	Komentar dan diskusi tugas.	Terhubung ke tugas dan user.
<code>task_dependencies</code>	Hubungan antar tugas.	Menjelaskan tugas mana bergantung pada tugas lain.

Contoh Relasi

```
users.id -> projects.pm_id
users.id -> tasks.assigned_to
projects.id -> tasks.project_id
projects.id -> milestones.project_id
tasks.id -> task_comments.task_id
tasks.id -> time_logs.task_id
```

9. Fitur Website dan Cara Fitur Dibuat

Fitur	Untuk Apa	Dibuat Dengan
Login	Membatasi akses hanya untuk akun terdaftar.	Login.jsx , AuthContext.jsx , auth.js , JWT, bcrypt, tabel users .
Dashboard	Menampilkan ringkasan proyek, tugas, progres, dan data penting.	Dashboard.jsx , API client, route API, tabel proyek dan tugas.
Manajemen Proyek	Tambah, lihat, edit, hapus proyek.	Form React, /api/projects , tabel projects .
Manajemen Tugas	Membuat tugas, memberi PIC, status, progress, dan due date.	Form React, /api/tasks , tabel tasks .
Kanban/Gantt Ringkas	Memudahkan melihat tugas berdasarkan status dan progres.	Data dari tasks , diolah di Dashboard.jsx .
Milestone Tracker	Melihat target pencapaian penting proyek.	/api/milestones , tabel milestones .
Tim	Mengelola nama tim dan anggota tim.	/api/teams , tabel teams dan team_members .
Quick Links	Menyimpan link repository, staging, BRD, atau API docs.	/api/project-links , tabel project_links .
Risk Register	Mencatat risiko, peluang, dampak, mitigasi, status, dan PIC.	/api/risks , tabel risks .
Risiko Delay Otomatis	Mendeteksi tugas terlambat atau mendekati tenggat.	Fungsi buildDelayRiskItems di frontend berdasarkan due date dan progress tugas.
Time Tracking	Mencatat berapa jam pekerjaan dilakukan.	/api/time-logs , tabel time_logs .
Resource Utilization	Melihat pemakaian jam kerja tiap anggota.	Data time_logs dihitung di frontend.
Burn-down Chart	Melihat sisa pekerjaan dari waktu ke waktu.	Chart.js dan data progress tugas.
File Repository	Menyimpan referensi dokumen proyek.	/api/project-files , tabel project_files .
Communication Log	Mencatat diskusi atau komentar tugas.	/api/task-comments , tabel task_comments .
Filter dan CSV	Menyaring tugas dan mengunduh data tugas.	State filter di Dashboard.jsx dan fungsi exportTasksCsv .

10. Cara Menggunakan Website

10.1. Membuka Website

1. Jalankan project dengan Docker: `docker compose up --build`.
2. Buka browser.
3. Masuk ke `http://localhost:8080`.
4. Halaman login akan tampil.

10.2. Login

Role	Username	Password
Project Manager	adminfairy	adminfairy
Developer	dev1	adminfairy
Pengguna proyek	client1	adminfairy

10.3. Membaca Dashboard

1. Lihat bagian ringkasan untuk jumlah proyek, tugas, completed, dan pending.
2. Lihat bagian proyek untuk status proyek dan progresnya.
3. Lihat bagian tugas untuk pekerjaan yang perlu dikerjakan.
4. Lihat grafik status tugas dan proyek untuk gambaran cepat.
5. Lihat risiko untuk mengetahui potensi hambatan.

10.4. Menambah Proyek

1. Login sebagai Project Manager.
2. Buka bagian **Kelola Data Proyek**.
3. Pilih tab proyek.
4. Isi nama proyek, deskripsi, tanggal, status, pengguna proyek, Project Manager, dan cover image bila ada.
5. Klik tombol simpan.

10.5. Menambah Tugas

1. Login sebagai Project Manager.
2. Pilih tab tugas.
3. Pilih proyek terkait.
4. Isi nama tugas, deskripsi, PIC, status, progress, dan due date.
5. Klik simpan. Tugas akan muncul di dashboard dan tabel tugas.

10.6. Update Progress Tugas

1. Login sebagai Project Manager atau Developer yang mendapatkan tugas.
2. Pilih tugas yang ingin diubah.

3. Ubah status menjadi `todo`, `in_progress`, atau `done`.
4. Ubah progress 0 sampai 100.
5. Simpan perubahan.

10.7. Mengisi Log Waktu

1. Buka tab log waktu.
2. Pilih tugas.
3. Masukkan jumlah jam kerja.
4. Pilih tanggal.
5. Klik simpan log.

10.8. Memberi Komentar Tugas

1. Buka tab komentar.
2. Pilih tugas.
3. Tulis komentar atau catatan pekerjaan.
4. Klik simpan komentar.

11. Penjelasan Fitur Visual dan Perhitungan

11.1. Chart.js

Chart.js dipakai untuk membuat grafik status tugas, status proyek, dan burn-down chart. Grafik membantu pembaca memahami data lebih cepat daripada membaca tabel panjang.

11.2. Burn-down Chart

Burn-down chart memperlihatkan sisa pekerjaan. Jika progress tugas naik, sisa pekerjaan turun. Ini membantu Project Manager melihat apakah proyek bergerak sesuai rencana.

11.3. Resource Utilization

Resource utilization dihitung dari tabel `time_logs`. Sistem mengelompokkan jam kerja berdasarkan user, lalu menghitung rata-rata jam per minggu.

11.4. Risiko Delay Otomatis

Fitur ini membaca tugas yang belum selesai. Jika due date sudah lewat, tugas dianggap terlambat. Jika due date dekat tetapi progress rendah, tugas diberi tanda risiko.

12. Keamanan Aplikasi

Keamanan	Dipakai Untuk	Lokasi Kode
Password Hash	Password tidak disimpan sebagai teks asli.	<code>bcrypt</code> di backend.
JWT	Token login untuk membuktikan identitas pengguna.	<code>auth.js</code> dan <code>middleware/auth.js</code> .
RBAC	Membatasi aksi berdasarkan role.	<code>authorizeRoles</code> .
Validasi Input	Mencegah data salah masuk ke database.	<code>schemas.js</code> .
CORS	Mengatur asal frontend yang boleh memanggil backend.	<code>app.js</code> dan <code>CORS_ORIGIN</code> .
Helmet	Menambah header keamanan HTTP.	<code>app.js</code> .

Catatan penting: akun demo dan password demo hanya cocok untuk presentasi atau pengujian. Untuk penggunaan sungguhan, ganti password, ganti `JWT_SECRET`, gunakan HTTPS, dan backup database.

13. Cara Menjalankan untuk Belajar

13.1. Dengan Docker

```
docker compose up --build
```

Alamat website: `http://localhost:8080`. Backend: `http://localhost:4000`. Health check: `http://localhost:4000/api/health`.

13.2. Manual Tanpa Docker

```
cd backend
npm install
npm run dev
```

```
cd frontend
npm install
npm run dev
```

Jika manual, MySQL harus aktif dan `database.sql` harus sudah diimport.

14. Alur Belajar yang Disarankan

1. Buka website dan coba login dengan akun demo.

2. Amati tampilan dashboard dan catat fitur yang terlihat.
3. Buka `frontend/src/pages/Login.jsx` untuk memahami form login.
4. Buka `frontend/src/context/AuthContext.jsx` untuk memahami penyimpanan token.
5. Buka `frontend/src/pages/Dashboard.jsx` untuk memahami tampilan fitur.
6. Buka `backend/src/app.js` untuk melihat route API dipasang.
7. Buka `backend/src/routes/projects.js` dan `tasks.js` untuk contoh CRUD.
8. Buka `database.sql` untuk memahami tabel dan relasi.
9. Coba tambah satu proyek, tambah satu tugas, lalu lihat perubahan di database atau dashboard.

15. Istilah Penting

Istilah	Arti Sederhana
Frontend	Tampilan yang dilihat pengguna.
Backend	Server yang memproses data dan aturan sistem.
Database	Tempat menyimpan data.
API	Jembatan komunikasi frontend dan backend.
Endpoint	Alamat API tertentu, misalnya <code>/api/tasks</code> .
JSON	Format data yang umum dikirim API.
State	Data sementara di frontend, misalnya isi form atau data dashboard.
Component	Bagian kecil tampilan React, misalnya halaman login atau tombol.
Middleware	Fungsi perantara di backend sebelum request masuk ke route utama.
Token	Bukti login digital.
RBAC	Pengaturan akses berdasarkan role.
Migration/Schema	Aturan pembuatan struktur database.

16. Checklist Pemahaman

- Saya paham bahwa frontend adalah tampilan, backend adalah server, dan database adalah penyimpanan.
- Saya paham login menghasilkan token yang dipakai untuk request berikutnya.
- Saya paham data proyek disimpan di tabel `projects` dan data tugas di tabel `tasks`.
- Saya paham fitur dashboard dibuat dari gabungan React, API backend, dan MySQL.
- Saya paham Project Manager punya akses lebih luas daripada Developer dan pengguna proyek.
- Saya paham cara membuka website, login, menambah proyek, menambah tugas, mengisi log waktu, dan memberi komentar.

Kesimpulan: portal ini adalah contoh aplikasi web full-stack. Setiap fitur yang terlihat di browser punya pasangan kode di frontend, API di backend, dan tabel di database. Dengan memahami tiga bagian itu, pembaca bisa mulai membaca, menjalankan, dan mengembangkan website secara bertahap.